CLESSOnet: ObsPair modelling for species richness and turnover via a Neural Network

Andrew Hoskins

April 2026

1 Background

This document describes a neural network implementation of the CLESSO (Community-Level Estimates from Sparse Species Observations) model. The core ecological framework – estimating species richness (α) and compositional turnover (β) from observation-pair data – is identical to the TMB version described in the companion document. The key difference is that the parametric sub-models (linear terms, P-splines, I-splines) are replaced with neural networks, while preserving the ecological constraints that make the model interpretable.

The motivation for this variant is computational. The TMB implementation relies on a Laplace approximation over site-level random effects (u_i), which requires repeated sparse Cholesky factorisations of the joint precision matrix. For large numbers of sites ($> 10^4$), this becomes prohibitively expensive – often requiring days to converge. A neural network trained with mini-batch stochastic gradient descent eliminates the Laplace approximation entirely, reducing training from days to minutes while retaining (and potentially improving upon) the flexibility of the spline-based approach.

Figure 1 provides an overview of the computational graph, showing how site-level covariates flow through the alpha, effort, and beta sub-networks to produce a match probability and composite loss.

2 Pair constructs and likelihood

We sample a set of observation pairs $P = P_W \cup P_S \cup P_N \cup P_D$ from species occurrence records using the hex-balanced sampler described in Section 3. For each pair p we record:

- i_p, j_p: the sites involved
- $y_p = 0$ if the species identities match, $y_p = 1$ otherwise
- d_p: a design weight correcting for unequal selection probability
- $s_p \in \{W, S, N, D\}$: the sampling stratum

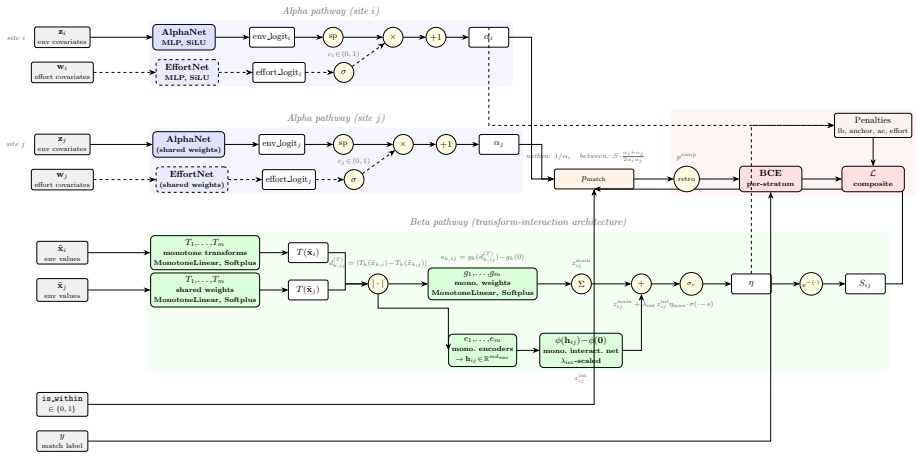

Figure 1: Computational graph of the CLESSO neural network forward pass (transform-interaction beta variant; Section 12.3). **Blue nodes**: alpha (richness) pathway — per-site environmental covariates $\mathbf{z}$ pass through a shared-weight AlphaNet (MLP) to produce $\alpha_i, \alpha_j > 1$ via softplus + 1. **Dashed nodes/arrows**: optional EffortNet pathway — effort covariates $\mathbf{w}$ produce a capture-completeness fraction $c \in (0,1)$ that modulates apparent richness (default: completeness mode, $\alpha^{\text{obs}} = 1 + c \cdot \text{softplus}(\text{env_logit})$). **Green nodes**: beta (turnover) pathway — per-site standardised environment vectors $\tilde{\mathbf{x}}_i, \tilde{\mathbf{x}}_j$ pass through shared-weight monotone transforms T_k, producing transformed separations $d^{(T)}_{k,ij} = |T_k(\tilde{x}_{k,i}) - T_k(\tilde{x}_{k,j})|$ (GDM I-spline analogue). **Main path**: per-dimension origin-anchored weighting functions g_k yield additive contributions $s_{k,ij} = g_k(d^{(T)}_{k,ij}) - g_k(0)$, summed to z^{main}. **Interaction path**: shallow monotone encoders e_k map each $d^{(T)}_{k,ij}$ to a low-dimensional embedding; the concatenated vector $\mathbf{h}_{ij} \in \mathbb{R}^{md_{\text{enc}}}$ is passed through a monotone interaction network ϕ with baseline subtraction, yielding a cross-dimension correction $z^{\text{int}}_{ij} = \phi(\mathbf{h}_{ij}) - \phi(\mathbf{0})$, scaled by λ_{int}. The combined predictor $z^{\text{main}} + \lambda_{\text{int}}\, z^{\text{int}}$ is mapped through a sigmoid clamp (σ_c) to turnover $\eta \in (0, \eta_{\max})$ and similarity $S = e^{-\eta}$. **Orange/red nodes**: likelihood and loss — the match probability p is computed from α_i, α_j, S, and `is_within`, corrected via the retrospective Bernoulli adjustment, and evaluated under per-stratum BCE with design weights. Penalty terms (lower-bound, anchor, anti-collapse, effort L_2) feed into the composite loss $\mathcal{L}$. The *additive* beta variant (Section 6.4) replaces the $T_k \rightarrow |\cdot| \rightarrow g_k$ chain with a single set of per-dimension monotone functions $f_k(|\Delta x_k|)$ operating directly on pre-computed distances. "sp" = softplus; "σ" = sigmoid; "σ_c" = sigmoid clamp $\eta_{\max} \cdot \sigma(\cdot - s)$; "retro" = retrospective Bernoulli correction.

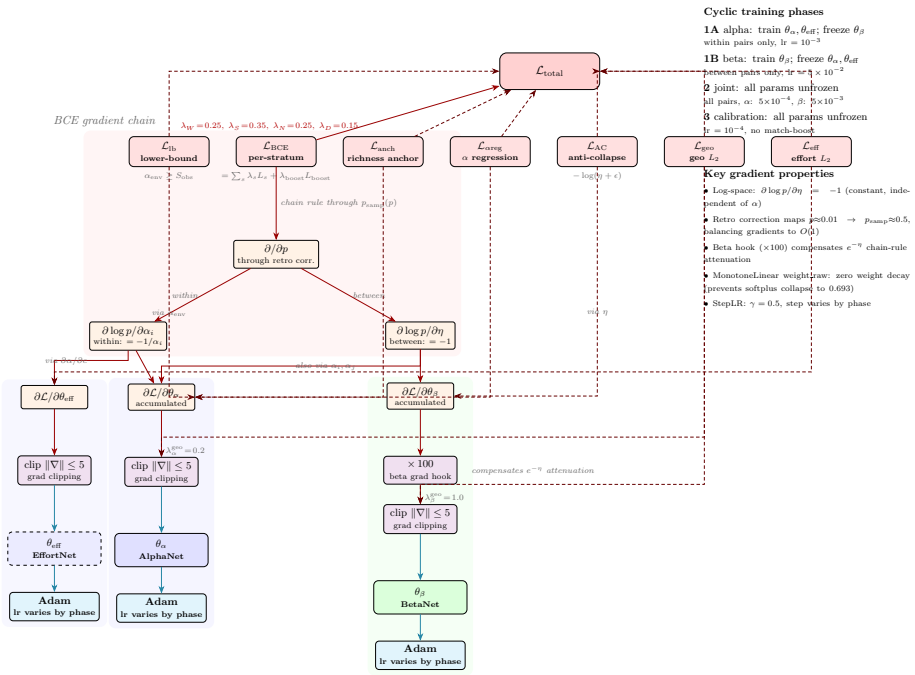

Figure 2: Computational graph of the CLESSO neural network backward pass (gradient flow). **Red/orange nodes**: gradient chain from the composite loss $\mathcal{L}$ through the per-stratum BCE, retrospective Bernoulli correction, and log-space match probability to the three parameter groups. Solid red arrows show the primary gradient flow from $\mathcal{L}_{\text{BCE}}$; dashed red arrows show penalty contributions ($\mathcal{L}_{\text{lb}}$, $\mathcal{L}_{\text{anch}}$, $\mathcal{L}_{\text{AC}}$, etc.). **Violet nodes**: gradient modifications — $\times 100$ backward hook on BetaNet parameters (compensating $e^{-\eta}$ attenuation) and global gradient clipping ($\|\nabla\|_2 \leq 5$). **Cyan nodes**: Adam optimisers with per-phase differential learning rates. The right panel summarises the cyclic block-coordinate training schedule: Phase 1A trains θ_α on within-site pairs while freezing θ_β; Phase 1B trains θ_β on between-site pairs while freezing θ_α; Phase 2 fine-tunes all parameters jointly with differential LR ($10\times$ higher for β); Phase 3 (optional) runs a short calibration pass. Key design property: the log-space formulation ensures $\partial \log p/\partial\eta = -1$ regardless of α magnitude, and the retrospective correction normalises match/mismatch gradient scales to $O(1)$.

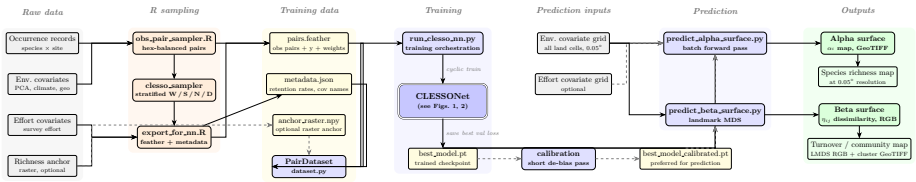

Figure 3: Full pipeline computational graph from sample generation to spatial prediction. **Grey**: raw input data (occurrence records, environmental and effort covariates, optional richness anchor raster). **Orange**: R sampling pipeline — `obs_pair_sampler.R` generates hex-balanced match/mismatch pairs, `clesso_sampler` stratifies them into within-site (W) and three between-site distance tiers (S, N, D), and `export_for_nn.R` writes the training artefacts. **Yellow**: training artefacts loaded into Python — `pairs.feather` (observation pairs with targets and design weights) and `metadata.json` (per-stratum retention rates and covariate names). **Blue**: training loop — `run_clesso_nn.py` orchestrates cyclic block-coordinate descent through the CLESSONet model (single node; internal architecture shown in Figs. 1 and 2); the best validation checkpoint is saved to disk (with an optional short calibration pass). **Green**: spatial prediction — `predict_alpha_surface.py` runs a batched forward pass over all land cells (0.05° grid) to produce a species richness GeoTIFF; `predict_beta_surface.py` selects landmark pixels, computes pairwise η, applies landmark MDS with Nyström extension, and writes an RGB turnover map and optional cluster GeoTIFF. Dashed arrows denote optional or secondary data flows (effort covariates, richness anchor, calibrated checkpoint).

Under the equal-draw assumption from Hoskins et al. (2020), the match probability decomposes as:

$$p_p = \begin{cases} \dfrac{1}{\alpha_i} & \text{if } i_p = j_p \text{ (within-site pair)} \\ S_{i,j} \cdot \dfrac{\alpha_i + \alpha_j}{2\,\alpha_i\,\alpha_j} & \text{if } i_p \neq j_p \text{ (between-site pair)} \end{cases} \tag{1}$$

where $\alpha_i > 1$ is the species richness at site i and $S_{i,j} = \exp(-\eta_{i,j}) \in [0, 1]$ is the Sørensen-type similarity derived from turnover $\eta_{i,j} \geq 0$.

3 Pair sampling

The sampling design directly influences model identifiability, computational efficiency, and the quality of the fitted turnover surface. Naïve random pair sampling from large occurrence databases produces a geographically biased sample: well-surveyed regions (e.g. Sydney Basin) contribute orders of magnitude more pairs per site than remote regions (e.g. northern Australia). Without correction, the model effectively ignores ecology in data-sparse regions.

The implementation uses a **hex-grid balanced sampler** that ensures every geographic region contributes training pairs across all spatial scales.

3.1 Hex-grid construction

A regular hexagonal grid of configurable cell size (default 2°) is overlaid on all site locations using `sf::st_make_grid(square=FALSE)`. Each site is assigned to its containing hex via spatial join. Only occupied hexes (those containing at least one site) are retained. For the Australian vascular plant dataset, this yields approximately 256 occupied hexes from ∼55,900 sites.

3.2 Sampling strata

Pairs are drawn into four strata, each with a 50/50 match:mismatch target quota:

Stratum W: Within-site Both observations from the same site. Anchors the richness model via $p = 1/\alpha_i$. Each hex receives an equal quota; within each hex, sites are sampled with probability proportional to m_i^2 (the number of possible within-site observation pairs), where m_i is the observed species count at site i.

Stratum S: Same-hex between-site Both sites in the same hex cell. Captures local beta diversity.

Stratum N: Neighbour-hex between-site Sites in adjacent hex cells (sharing an edge). Captures regional turnover (∼2–4°).

Stratum D: Distant-hex between-site Sites in non-adjacent hexes. Captures continental turnover and anchors asymptotic η.

An auxiliary **match-boost** set supplements the between-site sample by drawing species-stratified match pairs. These are tagged separately and handled as an auxiliary shaping loss (Section 4).

3.3 Design weights

Each pair carries a **design weight** $d_p = G(x_p)/q(x_p)$, where G is the target population measure and q is the known proposal probability under the sampler's scheme. These weights are fixed at sampling time and do not depend on model parameters.

Within-site design weights. For a pair from site i in hex h:

$$d_p^W = \frac{\sum_{j \in h} m_j^2}{n_h^{\text{elig}} \cdot m_i^2} \tag{2}$$

where n_h^{elig} is the number of eligible hexes and m_j is the observed species count at site j. This corrects for the equal-quota-per-hex scheme oversampling sparse hexes.

Between-site design weights. For a pair from site i in hex h_a and site j in hex h_b:

$$d_p^B = \frac{N_{h_a} \cdot N_{h_b}}{n_{h_a}^{\text{samp}} \cdot n_{h_b}^{\text{samp}} \cdot m_i \cdot m_j} \tag{3}$$

where N_h is the total number of observations in hex h and n_h^{samp} is the number of sampled observations from hex h within the tier. Design weights are normalised to mean 1 within each stratum.

4 Composite likelihood loss

4.1 Motivation

The sampler draws an approximately 50/50 match:mismatch ratio within each stratum, regardless of the true population frequency. This "quota sampling" produces a training set whose label distribution does not match the population. A naïve BCE loss would learn biased probabilities. Similarly, the four strata represent different pair populations with different ecological signals; their relative contribution to the loss must be explicitly controlled.

A key challenge is the extreme **gradient asymmetry** for between-site pairs. The population match probability for a between-site pair is $p = S \cdot (\alpha_i + \alpha_j)/(2\alpha_i\alpha_j)$. For typical richness values ($\alpha \sim 100$), $p \approx 0.005$–0.02. The BCE

match gradient ($-\log p$) is $\sim$200$\times$ larger in magnitude than the mismatch gradient ($-\log(1-p)$). With a 50/50 match:mismatch sample, the net gradient w.r.t. η is overwhelmingly driven by match pairs, pushing $\eta \to 0$ irrespective of the true turnover. Within-site pairs do not suffer this problem because both match and mismatch gradients scale as $1/\alpha$.

The loss therefore uses a **universal retrospective Bernoulli correction** across all strata, applying the same case–control adjustment (Equation 5) with per-stratum retention rates computed during sampling. For between-site pairs with $p \approx 0.01$, the retention-rate ratio ($r_0 \approx 1$, $r_1 \ll 1$) maps p to $p^{\text{samp}} \approx 0.5$, which naturally balances the match and mismatch gradient magnitudes without requiring ad-hoc class-balanced normalisation.

4.2 Per-stratum loss

The composite loss sums per-stratum BCE terms with explicit weights λ_s:

$$\mathcal{L}_{\text{main}} = \sum_{s \in \{W,S,N,D\}} \lambda_s \cdot L_s \tag{4}$$

Default stratum weights are $\boldsymbol{\lambda} = (0.25, 0.35, 0.25, 0.15)$ for (W, S, N, D), and $\ell(y, q) = -(1-y)\log q - y\log(1-q)$ is the binary cross-entropy. The per-stratum loss L_s is computed differently for within-site and between-site strata, as described below.

4.3 Within-site loss: retrospective Bernoulli correction

For within-site pairs (stratum W), the sampler's 50/50 quota means that the retention rate for matches ($r_{W,0}$) differs from that for mismatches ($r_{W,1}$). If the population match rate is p but the sampler retains matches at rate $r_{W,0}$ and mismatches at rate $r_{W,1}$, the probability of observing $y = 0$ in the sample is:

$$p^{\text{samp}} = \frac{r_{W,0} \cdot p}{r_{W,0} \cdot p + r_{W,1} \cdot (1-p)} \tag{5}$$

This is the standard retrospective likelihood correction for case–control designs. The retention rates are computed during sampling as the ratio of retained to drawn pairs for each class, and stored in the export metadata. For within-site pairs, the match and mismatch gradients w.r.t. α both scale as $1/\alpha$, so the retrospective correction produces balanced gradient signal without further adjustment.

$$L_W = \frac{\sum_{p \in P_W} d_p \cdot \ell(y_p, p_p^{\text{samp}})}{\sum_{p \in P_W} d_p} \tag{6}$$

When retention rates are unavailable (e.g. legacy exports), $r_{W,0} = r_{W,1} = 1$ is assumed, recovering the standard BCE.

4.4 Between-site loss: retrospective Bernoulli correction

For between-site pairs (strata S, N, D), the same retrospective Bernoulli correction used for within-site pairs is applied, but with per-tier retention rates. The sampler records separate retention rates $(r_{s,0}, r_{s,1})$ for each stratum s, reflecting the fraction of match and mismatch pairs retained from those drawn. The corrected probability is:

$$p_s^{\text{samp}} = \frac{r_{s,0} \cdot p}{r_{s,0} \cdot p + r_{s,1} \cdot (1-p)} \tag{7}$$

For a typical between-site pair with $p \approx 0.01$ and sampler quota yielding $r_0 \approx 1$, $r_1 \ll 1$ (say $r_1 \approx 0.01$), the correction maps $p^{\text{samp}} \approx 0.01/(0.01 + 0.01 \cdot 0.99) \approx 0.5$. At $p^{\text{samp}} \approx 0.5$, the match and mismatch BCE gradient magnitudes are equal, naturally resolving the gradient asymmetry that causes η collapse — without requiring explicit class-balanced normalisation or alpha-dependent weight corrections.

The between-site per-stratum loss has the same form as the within-site loss:

$$L_s = \frac{\sum_{p \in P_s} d_p \cdot \ell(y_p, p_{s,p}^{\text{samp}})}{\sum_{p \in P_s} d_p}, \quad s \in \{S, N, D\} \tag{8}$$

where d_p is the design weight (hex-IPW) and the BCE is evaluated on the corrected probability p^{samp} rather than the raw model prediction p. When retention rates are unavailable, $r_{s,0} = r_{s,1} = 1$ is assumed.

Anti-collapse regulariser. As a safety net against η collapse during early training (when retention rate estimates or the α model may be imprecise), an anti-collapse barrier term is added:

$$\mathcal{R}_\eta = -\lambda_{\text{ac}} \cdot \frac{1}{|P_{\text{btw}}|} \sum_{p \in P_{\text{btw}}} \log(\eta_p + \epsilon) \tag{9}$$

with $\lambda_{\text{ac}} = 0.1$ and $\epsilon = 10^{-4}$. The sum is over all between-site pairs in the batch. This term diverges as $\eta \to 0$, creating a soft lower bound that prevents degenerate solutions while adding negligible bias when η is at ecologically reasonable values ($\eta \gtrsim 1$). It replaces the earlier η^2 smoothness penalty, which inadvertently contributed to η collapse by penalising large values.

4.5 Match-boost auxiliary loss

Species-stratified match-boost pairs (stratum 4) are *not* part of the main composite likelihood because they are drawn from a different proposal distribution (species-conditional). Instead, they contribute an auxiliary shaping loss:

$$\mathcal{L}_{\text{boost}} = \lambda_{\text{boost}} \cdot \frac{1}{|P_{\text{boost}}|} \sum_{p \in P_{\text{boost}}} \ell(y_p, p_p) \tag{10}$$

with $\lambda_{\text{boost}} = 0.1$ by default. This helps the model see rare match pairs from distant sites without distorting the design-correct main loss.

Default: excluded. In the current default configuration, match-boost pairs are excluded entirely from training (`exclude_match_boost=True`). When excluded, stratum-4 pairs are removed from the pairs DataFrame during preprocessing so they never enter any dataloader or loss computation. The match-boost auxiliary loss is only active when `exclude_match_boost=False`.

4.6 Calibration phase

After the main training phases (cyclic env-only + fine-tune), a short **calibration phase** fine-tunes all parameters using only the design-weighted composite loss (no mining, no boost, no anchor penalties). This de-biases estimated probabilities and ensures the model's p predictions are well-calibrated for the population. Default settings: 50 epochs, learning rate 10^{-4}, patience 20 epochs. The calibrated model is saved separately as `best_model_calibrated.pt` and is preferred over the pre-calibration checkpoint by all downstream scripts (diagnostics, alpha/beta surface prediction).

4.7 Effective sample size

The effective sample size (ESS) is monitored per stratum:

$$\mathrm{ESS}_s = \frac{\left(\sum_{p \in P_s} \tilde{w}_p\right)^2}{\sum_{p \in P_s} \tilde{w}_p^2} \in [1, |P_s|] \tag{11}$$

For all strata, $\tilde{w}_p = d_p$ (design weights only), since the retrospective correction is applied to the predicted probability rather than the pair weight. The ESS is typically 50–70% of the stratum sample size.

5 Alpha sub-model: richness network

In the TMB model, richness is parameterised as:

$$\log(\alpha_i - 1) = \theta_0 + \mathbf{z}_i^\top \boldsymbol{\theta} + \sum_k g_k(z_{k,i}^{\mathrm{raw}}) + u_i$$

with linear terms, P-spline smooths $g_k(\cdot)$, and site-level random effects $u_i \sim \mathcal{N}(0, \sigma_u^2)$.

In the neural network variant, all of these components are replaced by a single multi-layer perceptron (MLP):

$$\alpha_i = \mathrm{softplus}(f_\alpha(\mathbf{z}_i; \boldsymbol{W}_\alpha)) + 1 \tag{12}$$

where f_α is a feedforward network with L hidden layers:

$$f_\alpha(\mathbf{z}) = \mathbf{W}^{(L+1)}\, \sigma\Big(\mathbf{W}^{(L)}\, \sigma\Big(\cdots \sigma\Big(\mathbf{W}^{(1)}\, \mathbf{z} + \mathbf{b}^{(1)}\Big) \cdots\Big) + \mathbf{b}^{(L)}\Big) + b^{(L+1)} \tag{13}$$

Here $\sigma(\cdot)$ is a nonlinear activation, and $\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}$ are the weights and biases at layer ℓ. The default architecture uses four hidden layers with dimensions $[256, 128, 64, 32]$ with SiLU (Sigmoid Linear Unit, $x \cdot \sigma(x)$) activation and dropout regularisation between layers. SiLU was chosen over ReLU for its smooth gradient landscape, which improves optimisation stability.

The output activation $\text{softplus}(\cdot) + 1$ guarantees $\alpha_i > 1$, matching the TMB constraint. The output bias is initialised so that $\text{softplus}(1.1) + 1 \approx 2.7$, providing a reasonable starting point for species richness. The key advantage is that the network simultaneously learns:

- the linear effects captured by $\mathbf{z}^\top \boldsymbol{\theta}$ in the TMB model
- the nonlinear smooth terms $g_k(\cdot)$ captured by P-splines
- the site-level heterogeneity captured by random effects u_i

all as a single differentiable function, without requiring the Laplace approximation needed by the random effects.

Note on random effects. The flexibility of the hidden layers largely subsumes what u_i captured in the TMB model. The P-spline basis with K knots per covariate provides $\mathcal{O}(K)$ degrees of freedom; the network provides $\mathcal{O}(\sum_\ell d_\ell \cdot d_{\ell+1})$ parameters, which is typically much larger. If additional site-level variation is needed, a learned per-site embedding could be added, though this risks overfitting.

5.1 Fourier positional encoding

Raw longitude and latitude coordinates are poor features for neural networks because they impose a linear geometry on what is a spherical surface. To provide the alpha network with multi-scale spatial information, we optionally replace (or supplement) raw coordinates with Fourier positional encodings:

$$\text{Fourier}_k(\text{lon}, \text{lat}) = \left[\sin\left(\frac{2\pi\,\text{lon}}{\lambda_k}\right),\ \cos\left(\frac{2\pi\,\text{lon}}{\lambda_k}\right),\ \sin\left(\frac{2\pi\,\text{lat}}{\lambda_k}\right),\ \cos\left(\frac{2\pi\,\text{lat}}{\lambda_k}\right)\right] \tag{14}$$

with wavelengths $\lambda_k = \lambda_{\max}/2^k$ for $k = 0, \ldots, N_f - 1$. With $\lambda_{\max} = 40°$ and $N_f = 5$, this produces wavelengths $[40, 20, 10, 5, 2.5]$ degrees, spanning continental to regional scales. Each frequency contributes 4 features (sin/cos for each coordinate), so $N_f = 5$ adds 20 features to $\mathbf{z}_i$.

This encoding allows the network to learn spatially structured richness patterns at multiple scales without requiring an explicit spatial random effect. When raw coordinates are excluded (`exclude_coords_from_alpha=True`), the Fourier features are the sole source of geographic information.

5.2 Alpha lower-bound penalty

A soft one-sided penalty discourages $\alpha_i^{\text{env}} < S_{\text{obs},i}$ (i.e. predicted true richness falling below observed species count). The penalty is applied to α^{env} (effort-

stripped true richness, not α^{obs}) because true richness must always be at least as large as the number of species actually recorded:

$$\mathcal{R}_\alpha = \frac{\lambda_{\text{lb}}}{|B|} \sum_{i \in B} \left[\frac{\log(1 + e^{\kappa(\log S_{\text{obs},i} - \log \alpha_i^{\text{env}})})}{\kappa} \right]^2 \tag{15}$$

where $\kappa = 10$ sharpens the hinge boundary and B is the current mini-batch. The penalty operates in log-space for scale invariance. Because the completeness model guarantees $\alpha^{\text{obs}} \leq \alpha^{\text{env}}$, penalising $\alpha^{\text{env}} < S_{\text{obs}}$ also ensures α^{obs} is anchored to a plausible true richness. Setting $\lambda_{\text{lb}} = 0$ disables the penalty entirely (current default).

5.3 Alpha anchor penalty

To prevent the runaway α inflation that kills beta gradients during block-coordinate training, a log-normal anchor penalty encourages the environment-only richness α_i^{env} (with effort zeroed) to stay within a tolerance of the observed species count $S_{\text{obs},i}$:

$$\mathcal{R}_{\text{anchor}} = \lambda_{\text{anchor}} \cdot \frac{1}{|B|} \sum_{i \in B} \max\Big(0,\ \big|\log \alpha_i^{\text{env}} - \log S_{\text{obs},i}\big| - \tau\Big)^2 \tag{16}$$

where $\tau = 0.7 \approx \log 2$ is a dead-zone tolerance and $\lambda_{\text{anchor}} = 1.0$. Within the dead zone ($\alpha_i^{\text{env}}$ between roughly $S_{\text{obs}}/2$ and $2S_{\text{obs}}$), the penalty is zero. Beyond it, a quadratic penalty pushes back. The penalty is applied to α^{env} (not α^{obs}) so that the EffortNet can independently explain the gap between true and apparent richness. Setting $\lambda_{\text{anchor}} = 0$ disables the penalty entirely.

5.4 Effort/detectability decomposition

Under the equal-draw assumption, α represents an *effective richness under the recording process* rather than true species richness. Consequently, spatial variation in sampling effort, observer intensity, institutional proximity, and road access can all inflate or deflate α estimates, creating artefactual structure that may be absorbed by the turnover model if not accounted for explicitly. To separate the observation-process signal from the ecological signal, the alpha model is decomposed into two additive components on the logit scale:

$$\alpha_i = \text{softplus}\Big(\underbrace{f_\alpha(\mathbf{z}_i;\, \boldsymbol{W}_\alpha)}_{\text{environment logit}} + \underbrace{f_\omega(\mathbf{w}_i;\, \boldsymbol{W}_\omega)}_{\text{effort logit}} \Big) + 1 \tag{17}$$

where $\mathbf{z}_i$ is the vector of environmental and substrate covariates (as before) and $\mathbf{w}_i$ is a separate vector of effort/detectability covariates at site i. The two sub-networks are:

EnvNet $f_\alpha(\mathbf{z})$: the existing alpha MLP (Section 3), unchanged in architecture. Its output is now interpreted as the pre-activation *environment logit*.

EffortNet $f_\omega(\mathbf{w})$: a smaller MLP (default hidden layers $[64, 32]$, SiLU activation, dropout) mapping effort covariates to a scalar *effort logit*. The output bias is initialised to zero so that the effort contribution is neutral at the start of training, i.e. $f_\omega(\mathbf{w}) \approx 0$ initially.

Three composition modes are supported, controlled by `effort_mode`:

"additive": $\alpha_i = \mathrm{softplus}(f_\alpha(\mathbf{z}_i) + f_\omega(\mathbf{w}_i)) + 1$.
Effort adds or subtracts effective species on the logit scale.

"multiplicative": $\alpha_i^{\mathrm{obs}} = (\mathrm{softplus}(f_\alpha(\mathbf{z}_i)) + 1) \cdot \sigma(f_\omega(\mathbf{w}_i))$.
Effort acts as a general $(0, 1)$ modifier on environment-predicted richness. Can push α below 1 for very low capture fractions.

"completeness" (default): Defines *true richness* $\alpha_i^{\mathrm{true}} = 1 + \mathrm{softplus}(f_\alpha(\mathbf{z}_i))$ and a *completeness fraction* $c_i = \sigma(f_\omega(\mathbf{w}_i)) \in (0, 1)$:

$$\alpha_i^{\mathrm{obs}} = 1 + c_i \cdot \mathrm{softplus}(f_\alpha(\mathbf{z}_i)) = 1 + c_i \cdot (\alpha_i^{\mathrm{true}} - 1). \tag{18}$$

Effort can only *reduce* apparent richness below true richness; $\alpha_i^{\mathrm{obs}} \geq 1$ is guaranteed. At initialisation $\sigma(0) = 0.5$, so $\alpha^{\mathrm{obs}} \approx 1 + \frac{1}{2}(\alpha^{\mathrm{true}} - 1)$. The completeness mode is algebraically similar to the multiplicative mode but with two key constraints: (i) the factor of 2 is absent, restricting c_i to be a strict down-scaling fraction, and (ii) the $+1$ is outside the product, ensuring $\alpha^{\mathrm{obs}} \geq 1$ regardless of the capture fraction. This makes it a proper *capture-completeness model*: c_i represents the fraction of true species that have been detected at site i, not a general multiplicative adjustment.

In either mode, the "true richness" at prediction time is obtained by zeroing the effort component.

The decomposition has several desirable properties:

1. **Clean separation at prediction time.** To predict true ecological richness (uncontaminated by sampling artefacts), we simply zero-out the effort logit: $\alpha_i^{\mathrm{env}} = \mathrm{softplus}(f_\alpha(\mathbf{z}_i)) + 1$. No EffortNet evaluation is needed.

2. **Diagnostic mappability.** The effort logit $f_\omega(\mathbf{w}_i)$ can be mapped spatially to visualise where sampling effort inflates or deflates apparent richness.

3. **Neutral initialisation.** Because f_ω starts near zero, the model begins training as if effort covariates were absent. The effort signal is learned only if it improves the likelihood.

4. **Backward compatibility.** When no effort covariates are supplied ($K_\omega = 0$), the EffortNet is not instantiated and the model reduces to the original architecture.

5. **Master switch.** A global `use_effort` flag (default `True`) can disable the EffortNet even when the dataset contains effort columns. When disabled, effort columns are still excluded from the alpha covariate set (preventing leakage into EnvNet) but no EffortNet is created.

Effort covariates. Six raster-derived effort layers are available at each site location:

Covariate	Description	Status
`ala_record_count`	Total ALA occurrence records per grid cell	*dropped*
`ala_record_smoothed`	Gaussian-smoothed record density	*dropped*
`dist_to_nearest_institution`	Distance (km) to nearest collection/museum	active
`hub_influence_unweighted`	Gravity-weighted influence of all institutions	active
`hub_influence_ecology_weighted`	As above, weighted by ecology-relevant collections	active
`road_density_km_per_km2`	Road network density (km road per km^2)	active

Endogenous covariate exclusion. After site×species deduplication, record-count covariates (`ala_record_count`, `ala_record_smoothed`) become endogenous: a site with 200 species surveyed once produces ∼200 records regardless of total survey effort. Feeding these into EffortNet gives it a noisy copy of the response variable (α), defeating the environment/effort decomposition. In multiplicative mode the problem is particularly acute: the network learns a capture fraction that correlates with apparent richness rather than true survey completeness, systematically inflating richness predictions in genuinely species-poor regions.

The `effort_drop_cov_names` config field (default: `["ala_record_count", "ala_record_smoothed"]`) removes these endogenous covariates from the EffortNet input while *still excluding them from the EnvNet covariate set*. This ensures dropped covariates cannot leak into either sub-network. The remaining four covariates (institutional proximity, hub influence, road density) are genuinely exogenous landscape properties independent of biology.

All layers share the same 0.05° grid as the environmental covariates (670×813 pixels). Values are standardised (zero-mean, unit-variance) before input to EffortNet.

Regularisation. An optional L_2 penalty on EffortNet parameters (controlled by `effort_penalty`) can be used to shrink the effort component toward zero, encouraging the model to explain as much variation as possible through the environmental pathway. The penalty is added to the total loss:

$$\mathcal{L}_{\text{effort}} = \lambda_\omega \sum_{\theta \in \boldsymbol{W}_\omega} \theta^2 \tag{19}$$

Interaction with training modes. The effort decomposition is active from the start of training in all modes (joint, two-stage, cyclic, cyclic fine-tune). During cyclic fine-tuning Phase 2, EffortNet weights are transferred from Phase 1 alongside the existing alpha architecture expansion.

6 Beta sub-model: monotone turnover network

6.1 TMB approach

In the TMB model, turnover is parameterised using I-spline basis functions with non-negative coefficients:

$$\eta_{i,j} = \exp(\eta_0^{\text{raw}}) + \sum_{l=1}^{m} \sum_{s=1}^{S} \exp(\beta_{l,s}^{\text{raw}}) \cdot |I_s(x_{l,i}) - I_s(x_{l,j})|$$

where $I_s(\cdot)$ are monotonically increasing I-spline bases and the exponentiation of the raw coefficients ensures $\beta_{l,s} \geq 0$. The composition of non-negative coefficients with monotone bases guarantees that $\eta_{i,j}$ is non-decreasing in each $|x_{l,i} - x_{l,j}|$.

6.2 Shared building block: MonotoneLinear layers

All beta network variants rely on a **MonotoneLinear** layer, which is a standard linear layer with the weight matrix constrained to be element-wise non-negative via a softplus transform on unconstrained parameters:

$$\tilde{\mathbf{W}}^{(\ell)} = \text{softplus}(\mathbf{W}_{\text{raw}}^{(\ell)}) \geq 0 \tag{20}$$

The unconstrained $\mathbf{W}_{\text{raw}}^{(\ell)}$ are the learnable parameters; the optimiser updates them freely while the softplus transform guarantees non-negative effective weights. The bias terms $\mathbf{b}^{(\ell)}$ are unconstrained.

Monotonicity proof. Consider the composition at any layer ℓ:

1. The input $\mathbf{x}^{(\ell-1)} \geq \mathbf{0}$ (base case: $\mathbf{\Delta} \geq 0$)
2. The linear map $\tilde{\mathbf{W}}^{(\ell)}\mathbf{x}^{(\ell-1)} + \mathbf{b}^{(\ell)}$ is non-decreasing in $\mathbf{x}^{(\ell-1)}$ because $\tilde{\mathbf{W}}^{(\ell)} \geq 0$
3. ReLU (or softplus) is monotone non-decreasing, so $\sigma(\cdot)$ preserves the ordering
4. By induction, the full composition is monotone non-decreasing in each input dimension

Critically, L_2 **weight decay must not be applied to $\mathbf{W}_{\text{raw}}$.** Standard L_2 regularisation penalises the raw parameters toward zero, but softplus(0) $= \ln 2 \approx 0.693$ for every weight, which drives all effective weights to the same value and causes *dimensional collapse* — all environmental dimensions receive identical weight regardless of their ecological importance.

6.3 Deep monotone network (`beta_type=‘‘deep’’`)

The simplest approach replaces the I-spline transform with a deep monotone MLP:

$$\eta_{i,j} = \mathrm{softplus}(h_\beta(\boldsymbol{\Delta}_{i,j}; \boldsymbol{W}_\beta)) \tag{21}$$

where $\boldsymbol{\Delta}_{i,j} = [|\tilde{x}_{1,i} - \tilde{x}_{1,j}|, \ldots, |\tilde{x}_{m,i} - \tilde{x}_{m,j}|] \geq \mathbf{0}$ is the vector of standardised environmental distances and h_β is a monotone MLP built from Monotone-Linear layers with ReLU activations.

This directly generalises the TMB approach: the I-spline + non-negative coefficient structure *is* a single-layer monotone network with a specific (spline) feature transform. The multi-layer version is a universal approximator for monotone functions (Daniels & Velikova, 2010). However, the deep variant is prone to dimensional collapse (all weights converging under weight decay) and is not the recommended architecture.

6.4 Additive monotone network (`beta_type=‘‘additive’’`)

The additive beta network is the direct neural analog of GDM's additive I-spline structure. Turnover decomposes as a sum of independent per-dimension monotone functions:

$$\eta_{i,j} = \sum_{k=1}^{m} f_k(|\tilde{x}_{k,i} - \tilde{x}_{k,j}|) \tag{22}$$

where each $f_k : \mathbb{R}_{\geq 0} \to \mathbb{R}$ is an independent monotone network with n hidden units (default $n = 32$):

$$g_k(d) = \mathbf{w}_2^\top \,\mathrm{softplus}(\tilde{\mathbf{w}}_1\, d + \mathbf{b}_1)\,, \qquad f_k(d) = g_k(d) - g_k(0) \tag{23}$$

Here $\tilde{\mathbf{w}}_1 = \mathrm{softplus}(\mathbf{w}_{1,\mathrm{raw}}) \geq 0$ and $\mathbf{w}_2 = \mathrm{softplus}(\mathbf{w}_{2,\mathrm{raw}}) \geq 0$ are non-negative weight vectors. The hidden activation is **softplus** rather than ReLU: softplus has no dead zone (always nonzero gradient), giving naturally gradual onset of turnover rather than the hard on/off threshold behaviour of ReLU. Softplus is still monotone non-decreasing, preserving the monotonicity guarantee.

The **baseline subtraction** $f_k(d) = g_k(d) - g_k(0)$ ensures that $f_k(0) = 0$ exactly, so zero environmental distance contributes zero turnover. Without this, the network biases would introduce a nonzero η even for identical sites.

The biases $\mathbf{b}_1$ in the first layer play the role of knot locations: they are initialised uniformly from -3σ to 0σ of the standardised environmental distance, spreading the response onset across the observed range. The second-layer weights are initialised very small (at $\mathrm{softplus}(-3) \approx 0.05$ effective weight) so that each dimension initially contributes ≈ 0.3 at maximum distance, giving a total initial $\eta \approx 5$ across all dimensions.

The final output is clamped: $\eta_{i,j} = \mathrm{clamp}(\sum_k f_k, 0, 10)$. The upper bound of 10 corresponds to $S = e^{-10} \approx 4.5 \times 10^{-5}$, effectively complete dissimilarity.

Without this clamp, the model can push $\eta \to \infty$ for all between-site pairs, learning the degenerate solution $p \approx 0$ everywhere.

6.5 Factored deep monotone network (`beta_type=``factored''`)

The factored architecture combines the per-dimension specialisation of the additive network with learned cross-dimension interactions:

$$\eta_{i,j} = \underbrace{\sum_{k=1}^{m} s_k(|\Delta x_k|)}_{\text{additive component}} + \underbrace{\phi(\mathbf{h}_1, \ldots, \mathbf{h}_m)}_{\text{interaction component}} \tag{24}$$

The architecture proceeds in three stages:

1. **Per-dimension monotone encoders**: Each $|\Delta x_k|$ (scalar) is mapped through a private multi-layer monotone encoder to a hidden representation $\mathbf{h}_k \in \mathbb{R}^{d_{\text{enc}}}_{\geq 0}$ (default $d_{\text{enc}} = 16$, depth 2). The final activation is ReLU, ensuring $\mathbf{h}_k \geq 0$ element-wise.

2. **Per-dimension scalar projections**: Each $\mathbf{h}_k$ is projected to a scalar $s_k = \mathbf{w}_k^\top \mathbf{h}_k$ via a MonotoneLinear$(d_{\text{enc}}, 1)$ layer, then baseline-subtracted: $s_k(\Delta x_k) - s_k(0)$. The sum $\sum_k s_k$ forms the additive component.

3. **Monotone interaction network**: The concatenation $[\mathbf{h}_1; \ldots; \mathbf{h}_m] \in \mathbb{R}^{m \cdot d_{\text{enc}}}$ is fed through a shared monotone MLP (MonotoneLinear layers with softplus activation) producing a scalar interaction term. This is also baseline-subtracted: $\phi(\mathbf{h}) - \phi(\mathbf{0})$.

Monotonicity guarantee: Each encoder is monotone in its input (non-negative weights + monotone activations). The interaction network has non-negative weights (MonotoneLinear) and takes non-negative inputs ($\mathbf{h}_k \geq 0$ via ReLU), so it is monotone in each $\mathbf{h}_k$. Since the composition of monotone functions is monotone, the full network is monotone in each $|\Delta x_k|$. Dropout is disabled throughout the factored network because baseline subtraction requires deterministic forward passes.

The interaction term allows the model to capture effects such as: the impact of temperature distance on turnover depends on the magnitude of precipitation distance. The additive network (Section 6.4) cannot represent such interactions.

6.6 Transform-first monotone network (`beta_type=``transform''`)

The transform beta network is the closest neural analog of GDM's I-spline structure. Where the additive and factored architectures operate on *pre-computed distances* $|\tilde{x}_{k,i} - \tilde{x}_{k,j}|$, the transform architecture first learns a monotone per-dimension transform of each site's raw (standardised) environmental value, and *then* takes the distance in transformed space:

$$\eta_{i,j} = \sigma_{\text{clamp}}\left(\sum_{k=1}^{m} g_k(|T_k(\tilde{x}_{k,i}) - T_k(\tilde{x}_{k,j})|)\right) \tag{25}$$

where $\sigma_{\text{clamp}}(z) = \eta_{\max} \cdot \sigma(z - s)$ is a sigmoid clamp with learnable shift s and maximum $\eta_{\max} = 10$, and each component is:

T_k**: per-dimension monotone transform.** A two-layer monotone network $T_k : \mathbb{R} \to \mathbb{R}$:

$$T_k(x) = \tilde{\mathbf{w}}_2^\top \operatorname{softplus}(\tilde{\mathbf{w}}_1\, x + \mathbf{b}_1) + b_2 \tag{26}$$

with $\tilde{\mathbf{w}}_1 = \operatorname{softplus}(\mathbf{w}_{1,\text{raw}}) \in \mathbb{R}_{\geq 0}^{n_T}$ and $\tilde{\mathbf{w}}_2 = \operatorname{softplus}(\mathbf{w}_{2,\text{raw}}) \in \mathbb{R}_{\geq 0}^{n_T}$, where n_T is the transform width (default 32). The composition of non-negative weights with a monotone activation (softplus) guarantees that T_k is monotonically non-decreasing. Dropout (default 0.1) is applied between layers for regularisation. Because T_k operates on the raw per-site value rather than a distance, it can discover nonlinear rescalings of the environmental axis — for example, that the difference between 10°C and 15°C matters ecologically more than the difference between 25°C and 30°C.

g_k**: per-dimension monotone weighting.** A two-layer monotone network $g_k : \mathbb{R}_{\geq 0} \to \mathbb{R}$ operating on the transformed distance $d_k = |T_k(\tilde{x}_{k,i}) - T_k(\tilde{x}_{k,j})|$:

$$g_k(d) = \tilde{\mathbf{v}}_2^\top \operatorname{softplus}(\tilde{\mathbf{v}}_1\, d + \mathbf{c}_1) \tag{27}$$

with n_g hidden units (default 16). The output is **origin-anchored** via baseline subtraction: the contribution of dimension k is $g_k(d_k) - g_k(0)$, ensuring zero distance contributes zero turnover. This mirrors the additive network's baseline subtraction (Section 6.4).

Relationship to GDM I-splines. In GDM, turnover is parameterised as $\eta_{i,j} = \sum_k \sum_s \beta_{k,s}\, |I_s(x_{k,i}) - I_s(x_{k,j})|$, where I_s are monotone I-spline basis functions and $\beta_{k,s} \geq 0$. The transform network replaces the fixed I-spline bases $\{I_s\}$ with a *learned* monotone transform T_k, and replaces the linear weighting $\sum_s \beta_{k,s}$ with a learned monotone weighting function g_k. The structural correspondence is:

GDM I-spline	Transform NN
Fixed I-spline bases $I_s(\cdot)$	Learned monotone $T_k(\cdot)$
Non-neg coefficients $\beta_{k,s}$	Learned monotone $g_k(\cdot)$
Linear combination of basis diffs	Nonlinear weighting of transform diffs
Knot placement chosen a priori	"Knot" positions learned (biases $\mathbf{b}_1$)

Monotonicity guarantee. The guarantee follows from three facts: (1) T_k is monotone non-decreasing (MonotoneLinear + softplus composition); (2) if $|x_{k,i} - x_{k,j}|$ increases and T_k is monotone, then $|T_k(x_{k,i}) - T_k(x_{k,j})|$ is non-decreasing; (3) g_k is monotone non-decreasing with $g_k(0) = 0$ via baseline subtraction. The sum of non-negative monotone terms is itself monotone and non-negative, so η is monotone non-decreasing in each $|x_{k,i} - x_{k,j}|$.

Output activation. Unlike the additive and factored networks which use a hard clamp, the transform network uses a **sigmoid clamp**: $\eta = \eta_{\max} \cdot \sigma(\text{raw} - s)$, where s is a learnable shift parameter calibrated at initialisation so that a typical pair ($\sim 0.5\sigma$ apart in each dimension) yields $\eta \approx 1.5$. The sigmoid provides smooth, everywhere-nonzero gradients, avoiding the dead gradient zone at the upper clamp boundary that can trap the additive/factored networks.

Data flow. The transform network requires per-site environmental vectors $(\mathbf{x}_i, \mathbf{x}_j)$ as input, rather than the pre-computed distance vector $\mathbf{\Delta}_{i,j}$ used by the additive and factored networks. The dataset pipeline retains the raw (standardised) per-site environmental values alongside the pre-computed distances, so both pathways are available. For within-site pairs ($i = j$), the distances are identically zero by construction.

6.7 Geographic distance as a beta input

Optionally, the beta network can include geographic (Haversine) distance as an additional input dimension:

$$\mathbf{\Delta}_{i,j} = \left[|\tilde{x}_{1,i} - \tilde{x}_{1,j}|, \ldots, |\tilde{x}_{m,i} - \tilde{x}_{m,j}|, \ \frac{d_{\text{geo}}(i,j)}{\sigma_{\text{geo}}} \right] \tag{28}$$

where $d_{\text{geo}}(i,j)$ is the great-circle distance in kilometres and σ_{geo} is estimated from random site pairs. This additional dimension gets its own monotone subnetwork (additive or encoded) and allows the model to capture *distance decay* — the empirical observation that community similarity decreases with geographic distance independent of environmental differences (e.g. dispersal limitation).

The geographic distance feature is typically introduced in Phase 2 of the cyclic fine-tuning procedure (Section 7.2.4), after the environmental signal has been learned.

6.8 Comparison

	TMB (I-spline)	Additive NN	Transform NN	Factored NN
Monotonicity	Non-neg coef × I-spline	Non-neg weights × softplus	Non-neg weights × softplus	Non-neg wei
Input	Per-site $x_{k,i}$	Pre-computed $\lvert\Delta x_k\rvert$	Per-site $x_{k,i}$	Pre-compute
Flexibility	$S \times m$ params	$n \times m$ params	$(n_T + n_g) \times m$ params	$n \times m$ + inte
Interactions	None (additive)	None (additive)	None (additive)	Learned via
$f_k(0) = 0$	By construction	Baseline subtraction	Baseline subtraction	Baseline sub
η range	$[0, \infty)$	$[0, 10]$ clamped	$(0, 10)$ sigmoid	$[0, 10]$ clamp
Collapse risk	Low	Low	Low	Low (per-din
GDM analogy	Identical structure	Learned f_k on distance	Closest analog	Extended wi

The factored network is the default (`beta_type=''factored''`) as it combines per-variable interpretability with the ability to capture cross-dimension interactions. The additive variant (`beta_type=''additive''`) is available when

strict additivity and maximal interpretability are preferred. The transform variant (`beta_type=''transform''`) is the closest neural analog of GDM's I-spline parameterisation and is preferred when the goal is to recover ecologically interpretable per-dimension transforms comparable to GDM partial functions.

6.9 Discussion: similarities and differences between the four beta architectures

All four beta architectures — including the TMB I-spline baseline — share the same fundamental structure: turnover $\eta_{i,j}$ is a non-negative, monotone non-decreasing function of the environmental separation between sites i and j, decomposed additively across covariate dimensions (with the factored network adding an interaction term on top). They all guarantee $\eta \geq 0$ and monotonicity in each $|x_{k,i} - x_{k,j}|$, and they all produce ecologically interpretable per-dimension response curves showing how turnover accumulates with environmental distance. The differences lie in *where* the nonlinear transform is applied and *what* the network takes as input.

Distance-first vs. transform-first. The additive and factored networks are **distance-first**: the data pipeline pre-computes environmental distances $|\tilde{x}_{k,i} - \tilde{x}_{k,j}|$ and the network learns a monotone function of those distances. The transform network is **transform-first**: it receives the raw per-site environmental values $(\tilde{x}_{k,i}, \tilde{x}_{k,j})$, applies a learned monotone transform T_k to each, and *then* takes the distance in transformed space. The TMB I-spline approach is also transform-first — $|I_s(x_{k,i}) - I_s(x_{k,j})|$ — making the transform NN the closest neural analog of GDM.

This distinction matters ecologically. The distance-first approach treats every unit of standardised environmental distance the same: a 1σ difference in temperature contributes the same raw input regardless of whether the pair is at 5°C vs. 6°C or at 30°C vs. 31°C. The learned monotone function f_k can then up-weight or down-weight *different magnitudes of distance*, but it cannot distinguish *where on the environmental axis* the distance occurs.

The transform-first approach can make this distinction. If T_k compresses the upper end of the temperature axis (i.e. $T_k'(x)$ is small for large x), then a 1°C difference between warm sites maps to a smaller transformed distance than the same 1°C difference between cool sites. This is precisely the behaviour of GDM's I-splines, where the spline basis functions concentrate their "knots" (regions of rapid change) in the part of the environmental axis where species turnover is most sensitive.

Parameters and capacity. For a model with m environmental dimensions:

- The **additive NN** has $\sim 2n \cdot m$ parameters (one two-layer monotone network per dimension, with n hidden units).

- The **transform NN** has $\sim 2(n_T + n_g) \cdot m$ parameters: a T_k network (n_T hidden units) plus a g_k network (n_g hidden units) per dimension. With defaults $n_T = 32, n_g = 16$, this is roughly $3\times$ the additive network.
- The **factored NN** has $\sim 2n \cdot m$ per-dimension encoder parameters plus the shared interaction network. The interaction network scales as $\mathcal{O}(m \cdot d_{\text{enc}} \cdot d_{\text{hidden}})$, so total capacity grows superlinearly with dimension count.
- The **TMB I-spline** has $S \times m$ parameters (one coefficient per spline basis per dimension), typically 8–20 basis functions.

Interpretability. The additive and transform networks are the most interpretable: each dimension's contribution can be visualised independently by sweeping the input while holding other dimensions at zero. For the additive network this produces response curves $f_k(d)$ showing turnover as a function of environmental distance. For the transform network, one can additionally visualise the T_k transform itself, showing how the raw environmental axis is nonlinearly rescaled — directly analogous to plotting GDM's fitted I-spline partial functions. The factored network's additive component is equally interpretable, but the interaction component requires joint response surfaces (two-dimensional sweeps) to visualise, making it harder to summarise succinctly.

Gradient behaviour. The additive and factored networks use a hard clamp ($\text{clamp}(\cdot, 0, 10)$) for the output, which has zero gradient when η hits the boundary. In practice, pairs near the upper boundary can become "stuck" with no gradient signal to refine their turnover estimate. The transform network uses a sigmoid output activation $\eta_{\max} \cdot \sigma(\cdot - s)$, which always has nonzero gradient regardless of the η value, providing smoother optimisation dynamics at the cost of never reaching exactly $\eta = 0$ or $\eta = 10$.

When to use each.

- **Additive**: simplest, fastest, most interpretable. Preferred when there are many environmental dimensions and strict per-variable additivity is desired. Cannot represent cross-dimension interactions or position-dependent sensitivity.
- **Transform**: closest to GDM, position-aware, fully interpretable. Preferred when the goal is to recover ecologically meaningful per-dimension transforms and to allow position-dependent sensitivity along environmental axes. Somewhat more expensive due to two forward passes per dimension (one per site). Like the additive network, it cannot represent cross-dimension interactions.
- **Factored**: most flexible, captures interactions. Preferred when cross-dimension effects are expected (e.g. temperature–precipitation interactions) and some interpretability can be traded for capacity. The per-

dimension encoders retain variable-level interpretability for the additive component.

- **Deep**: included for completeness but not recommended. Prone to dimensional collapse under weight decay and offers no per-variable interpretability.

7 Optimisation

The TMB model is fitted via maximum marginal likelihood using `nlminb`, with the Laplace approximation integrating over the random effects $\mathbf{u}$. Each function evaluation requires a sparse Cholesky factorisation of the joint precision matrix, which is $\mathcal{O}(n)$ in the number of sites but with a large constant.

The neural network is fitted via mini-batch stochastic gradient descent using Adam (Kingma & Ba, 2015). Each parameter update costs $\mathcal{O}(B \cdot D)$ where B is the batch size and D is the total number of network parameters. There is no Hessian or Laplace approximation.

	TMB	Neural network
Optimiser	`nlminb` (quasi-Newton)	Adam (adaptive SGD)
Random effects	Laplace approximation	None (absorbed by network)
Per-iteration cost	Sparse Cholesky ($\sim$ seconds)	Matrix multiplies ($\sim$ milliseconds)
Typical runtime	Hours to days	Minutes
Hardware	CPU only	CPU, GPU, or Apple MPS

Training details:

- **Learning rate scheduling**: Cosine annealing with warm restarts ($T_0 =$ 30 epochs, $T_{\text{mult}} = 2$), giving cycle lengths of 30, 60, 120,... epochs. The minimum LR is 10^{-6}.
- **Early stopping**: halt training if validation loss does not improve for a configurable number of epochs (default 40 for full-phase training; 5 within each beta phase of cyclic training).
- **Gradient clipping**: max norm of 5.0 on all parameters to prevent exploding gradients.
- **Weight decay**: L_2 penalty of 10^{-5} on alpha parameters and beta bias parameters. *No weight decay* on beta `weight_raw` parameters (see Section 6.4).
- **Dropout**: applied between hidden layers (default rate 0.1) to reduce overfitting.
- **Batch size**: 8192 pairs per mini-batch (default).

7.1 Gradient imbalance between alpha and beta

A fundamental challenge in jointly training the alpha and beta sub-models is a severe gradient imbalance. The match probability for between-site pairs is:

$$p_{\text{between}} = \underbrace{e^{-\eta}}_{S} \cdot \frac{\alpha_i + \alpha_j}{2\,\alpha_i\,\alpha_j}$$

Differentiating the log-likelihood with respect to η introduces a factor of $\partial p/\partial \eta = -p$, so beta gradients are attenuated by a factor of approximately $p \sim 1/\alpha$. For realistic richness values ($\alpha \approx 3$–100), beta gradients can be 100–1000× smaller than alpha gradients, effectively starving the turnover network of learning signal.

Three mechanisms address this imbalance:

1. **Beta LR multiplier**: The beta network receives a higher base learning rate than alpha (default 50×), implemented via separate Adam parameter groups.

2. **Gradient amplification hooks**: PyTorch backward hooks on beta parameters scale their gradients by a configurable factor (default 100× in joint training, 1× in cyclic/two-stage modes where the retrospective correction provides sufficient signal). This is applied *before* gradient clipping.

3. **Retrospective Bernoulli correction**: All strata use case–control retrospective correction (Section 4.4), mapping the model's predicted p to a sampling-corrected p^{samp} using per-stratum retention rates from the sampler. This naturally balances match and mismatch gradient magnitudes without ad-hoc weight manipulation.

7.2 Training modes

Four training modes are implemented, addressing the alpha–beta gradient imbalance with increasing sophistication:

7.2.1 Joint training (`mode=``joint''`)

All parameters are optimised simultaneously on all pairs. This is the simplest mode but is prone to *beta collapse*: alpha inflates to match sampled pair frequencies rather than true richness, making $\partial p/\partial \eta \approx 0$ and leaving beta unable to learn. Gradient amplification and the beta LR multiplier partially mitigate this, but joint training is not recommended for production use.

7.2.2 Two-stage training (`mode=``two_stage''`)

Mirrors the original CLESSO two-stage approach (`clesso_alpha.cpp` → `clesso_beta_fixAlpha.cpp`):

Stage 1: Freeze beta, train alpha on *within-site pairs only* with BCE + lower-bound penalty. Alpha converges to realistic richness values ($\alpha \approx S_{\text{obs}}$).

Stage 2: Freeze alpha, train beta on *between-site pairs only* with importance-weighted BCE. Beta receives uncontested, balanced gradients and learns non-trivial turnover response curves.

Each stage uses its own optimizer (Adam with cosine annealing) and early stopping.

7.2.3 Cyclic block-coordinate descent (`mode=''cyclic''`)

Mirrors `clesso_iterative.R`: alternating block-coordinate optimisation with damped step sizes:

for cycle $= 1, \ldots, C_{\max}$: (A) freeze β, train α for N_α epochs (B) freeze α, train β for N_β epochs

Default settings use $N_\alpha = 5$ and $N_\beta = 20$ (small alpha steps, $4\times$ beta steps), keeping alpha changes incremental so beta can adapt without oscillation. Each beta phase also has its own **within-phase patience** (default 5 epochs): if the beta validation loss does not improve for 5 consecutive epochs within a cycle, that beta phase terminates early rather than running all N_β epochs. This prevents wasted computation when beta has converged for the current alpha state. Convergence across cycles is declared when the relative change in best beta validation loss between cycles falls below a threshold (10^{-4}).

Key differences from two-stage:

- Alpha and beta *see each other's updated parameters repeatedly*, allowing co-adaptation.
- Alpha can re-adjust after beta has learnt turnover (e.g. correcting inflated estimates).
- Persistent optimiser state (momentum) is maintained across cycles.
- Step-based LR decay (halve every 200 global epochs) replaces per-stage cosine annealing.

7.2.4 Cyclic + fine-tuning (`mode=''cyclic_finetune''`)

This is the **default training mode**. A two-phase procedure where Phase 1 learns the core ecological signal via damped cyclic training, and Phase 2 refines the fit with a configurable fine-tuning strategy.

Phase 1: Damped cyclic training on *environmental and substrate covariates only* (no Fourier features, no geographic distance). This learns the core ecological signal without geographic confounding.

Phase 2: Controlled by `finetune_mode`:

"joint" (default): Joint alpha+beta training on the *same* environmental/effort variables used in Phase 1. No model expansion. Both sub-networks are trainable with separate learning rates (alpha at `finetune_lr`, beta at `finetune_lr` × `finetune_joint_beta_lr_mult`, default 10×). This allows the two sub-models to co-adapt after the alternating Phase 1 has stabilised their individual domains.

"geo": Expand the model architecture to accommodate new geographic input dimensions: Fourier positional encoding for alpha (zero-initialised weights so Phase 1 predictions are initially unchanged) and haversine distance for beta (new per-dimension monotone sub-network). Fine-tune with a smaller learning rate. Optionally, Phase 1 parameters can be frozen (`finetune_freeze_existing=True`) so only the new geographic parameters are trained. Strong L_2 penalties on geographic parameters (`geo_penalty_alpha`, `geo_penalty_beta`) ensure the model only uses spatial information when site-level covariates cannot explain the pattern.

The two-phase structure mirrors the intuition from spatial ecology that environmental variables should explain most compositional turnover, with Phase 2 refining the joint fit or adding geographic residual signal as needed.

7.3 Hard-pair mining with importance correction

The pairwise likelihood in equation (2) is defined with respect to a *target pair distribution* over the sampled pair set P, represented by the hex-IPW weights w_p (Section **??**). Naïve hard-pair mining changes this training distribution by oversampling difficult pairs, which can improve optimisation speed but distort the resulting fitted probabilities. To retain the original probabilistic target while focusing optimisation on informative pairs, hard-pair mining is introduced as a *proposal distribution* with importance correction.

Target and proposal distributions. Let $\pi(p)$ denote the target sampling probability for pair p, corresponding to the original pair-construction design, and let $q(p)$ denote the actual probability that pair p is selected into a mini-batch under the mining scheme. The training loss uses an importance-corrected weight:

$$w_p^\star = w_p \cdot \frac{\pi(p)}{q(p)} \tag{29}$$

In practice, multiple layers of importance correction are composed. For a between-site pair in a mined mini-batch, the effective weight is:

$$\tilde{w}_p^{\text{eff}} = \underbrace{d_p}_{\substack{\text{hex-IPW}\\ \text{weight}}} \cdot \underbrace{\frac{\pi(p)}{q(p)}}_{\substack{\text{mining}\\ \text{correction}}} \tag{30}$$

where d_p is the design weight (hex-IPW) from Section **??**. The mining ratio corrects for the non-uniform mini-batch sampling introduced by difficulty-based re-sampling. When $q(p) = \pi(p)$, the mining correction is unity and the loss reduces to the standard retrospectively corrected likelihood.

The mini-batch loss is:

$$\widehat{\mathcal{L}}_B = -\frac{1}{\sum_{p\in B} \tilde{w}_p^{\text{eff}}} \sum_{p\in B} \tilde{w}_p^{\text{eff}} \Big[(1-y_p)\log p_p + y_p \log(1-p_p)\Big] + \lambda_{\text{lb}} \cdot \mathcal{R}_\alpha \quad (31)$$

where $\tilde{w}_p^{\text{eff}}$ is the design weight from Section **??** further corrected by the mining importance ratio.

Coarse pair strata. Hard-pair mining is applied only to *between-site* pairs during beta training phases, since within-site pairs anchor the alpha model and have a fundamentally different loss surface ($p = 1/\alpha$, no η dependence). Between-site pairs are divided into two coarse cells:

$$c(p) \in \{\text{between-match, between-mismatch}\}$$

Each cell receives equal target mass, so the target distribution is:

$$\pi(p) = \Pr(c(p)) \cdot \frac{1}{N_{c(p)}} = \frac{1}{2} \cdot \frac{1}{N_{c(p)}} \quad (32)$$

where N_c is the number of training pairs in cell c.

Difficulty strata. For each between-site pair p, a hardness score is computed using the current per-pair negative log-likelihood:

$$h_p = -\Big[(1-y_p)\log p_p + y_p \log(1-p_p)\Big] \quad (33)$$

Within each coarse cell, pairs are ranked by h_p and partitioned into D difficulty bins (default $D = 3$: easy, medium, hard) using quantile boundaries. Let $d(p)$ denote the difficulty bin for pair p, and $N_{c,d}$ the number of pairs in cell c and bin d.

Using quantile-based bins rather than absolute NLL thresholds avoids a subtle issue: pairs near the $\eta = 10$ clamp boundary (Section 6.4) have irreducibly high NLL due to the modelling constraint, not because they are informative. Quantile ranking within each cell ensures these pairs are distributed across bins rather than dominating the "hard" bin.

Mixture proposal. The proposal distribution for between-site pairs is a mixture:

$$q(p) = (1-\lambda_{\text{hm}})\,\pi(p) + \lambda_{\text{hm}}\, h^\star(p) \quad (34)$$

where $\lambda_{\text{hm}} \in [0,1]$ controls the mining strength and $h^\star(p)$ is the hard-pair proposal:

$$h^\star(p) = \Pr(c(p)) \cdot \Pr(d(p) \mid c(p)) \cdot \frac{1}{N_{c(p),d(p)}} \quad (35)$$

The conditional bin probabilities $\Pr(d \mid c)$ assign extra mass to difficult bins (default: $[0.2, 0.3, 0.5]$ for easy/medium/hard). At $\lambda_{\text{hm}} = 0$ (default), the proposal equals the target and all mining code is a no-op.

Variance control. The importance ratio $\pi(p)/q(p)$ can become large when easy pairs are drawn from the hard-biased proposal. To prevent unstable gradients, the ratio is truncated:

$$\frac{\pi(p)}{q(p)} \leftarrow \min\left(\frac{\pi(p)}{q(p)},\ a_{\max}\right) \tag{36}$$

with $a_{\max} = 5$ by default. This introduces a small bias in exchange for substantially reduced gradient variance. The truncation interacts with the existing $1/\alpha_h$ importance weighting: high-α match pairs from easy bins receive the product of both corrections, so the truncation cap prevents doubly extreme weights.

Practical training scheme. Training proceeds in two phases:

1. **Warm-up.** For the first few cycles (default 3), mini-batches are sampled from the target distribution only ($\lambda_{\text{hm}} = 0$). This avoids using unstable model-based hardness scores early in training when the beta network has not yet learned meaningful turnover.

2. **Mining phase.** At the start of each subsequent beta training phase (Phase B of cyclic training, or each epoch in two-stage mode), a forward pass over all between-site training pairs computes hardness scores h_p, refreshes the difficulty bins, and rebuilds the proposal distribution $q(p)$. Scores can optionally be refreshed mid-phase (default: every beta epoch) to track the evolving loss landscape.

The scoring forward pass is inexpensive — it amounts to one epoch of inference over the training pairs with no gradient computation — and is easily amortised over the subsequent training epochs.

Interpretation. This scheme preserves the original probabilistic target of the weighted Bernoulli likelihood while allowing optimisation to focus more heavily on informative between-site pairs. In effect, hard-pair mining is treated as a *sampling design for stochastic optimisation* rather than a change to the statistical model itself. The fitted match probabilities remain calibrated with respect to the original pair distribution, which is critical because downstream predictions (beta-diversity indices, compositional similarity maps) rely on these probabilities being interpretable rather than merely ordinal.

8 Relationship to TMB model

The neural network variant is not a different model in the ecological sense. The likelihood (equation 2), the match probability decomposition (equation 1), and

the pair sampling strategy are all identical. What changes is the function class used to map covariates to α and environmental distances to η:

Component	TMB	Neural network
α_i	Linear + P-spline + random effect	MLP (SiLU) with softplus output
Effort separation	Not modelled	Additive/multiplicative EffortNet
$\eta_{i,j}$	I-spline + non-negative coefficients	Additive/factored monotone network
p_p	Same formula	Same formula
$\mathcal{L}$	Same weighted Bernoulli NLL	Same (+ retrospective correction)
$\mathcal{R}_\alpha$	Same softplus^2 lower-bound penalty	Same softplus^2 lower-bound penalty
Training	`nlminb`, two-stage, iterative	Adam, two-stage/cyclic block-coordinate
Geography	I-spline on geo distance	Fourier encoding (α) + haversine dim (β)

The neural network can be viewed as replacing the manually specified basis functions (B-splines, I-splines) with learned feature representations, and replacing the Gaussian random effects with implicit regularisation through dropout and weight decay. The additive beta network (default) preserves the per-variable interpretability of GDM while gaining the flexibility of learned (rather than pre-specified) knot placements.

9 Post-training: diagnostics and spatial prediction

After training, the pipeline automatically runs three post-processing steps:

9.1 Diagnostics

A comprehensive diagnostic PDF (mirroring the R script `clesso_diagnostics.R`) is generated, containing:

- Training curves (loss, accuracy, alpha statistics, learning rate schedule)
- Residual analysis (within-site and between-site)
- Observed vs. predicted match probability
- Classification metrics (AUC, confusion matrix)
- Spatial map of fitted α (richness)
- α vs. observed species richness comparison
- Beta turnover response curves per environmental dimension (analogous to I-spline partial functions)
- Similarity decay curves $S = \exp(-\eta)$
- Alpha partial-dependence plots (site covariates $\rightarrow$ richness)
- Variable importance bar chart

9.2 Alpha surface

Species richness is predicted across all of Australia using the trained alpha network applied to a gridded covariate stack (0.05° resolution, 670×813 cells). The covariates include 6 substrate PCA bands, 9 AWAP climate variables (matching training columns), and optionally Fourier positional features. Output files are saved to the same directory as the model checkpoint.

When the effort decomposition is active (Section 5.4), three alpha surfaces are produced:

1. **Environment-only** (α^{env}): the primary prediction surface, computed as $\text{softplus}(f_\alpha(\mathbf{z})) + 1$ with the effort logit zeroed. This represents estimated true richness uncontaminated by sampling artefacts and is saved as the main GeoTIFF.

2. **Full** (α^{full}): the combined prediction $\text{softplus}(f_\alpha(\mathbf{z}) + f_\omega(\mathbf{w})) + 1$, representing apparent richness as influenced by both environment and observation process. Requires effort raster layers at the same grid resolution.

3. **Effort logit** ($f_\omega(\mathbf{w})$): the raw effort network output, mapped spatially to visualise where sampling effort inflates (positive values) or deflates (negative values) apparent richness relative to the environmental baseline.

All three are saved as separate GeoTIFF files.

9.3 Beta surface (community composition map)

A spatial map of biological community composition is generated using Landmark MDS, with output saved alongside the model checkpoint:

1. Select k landmark pixels via spatial k-means (default $k = 500$)

2. Compute pairwise η between all landmark pairs using the trained beta network ($k \times k$ matrix)

3. Compute η from every pixel to each landmark ($n \times k$ matrix)

4. Convert to dissimilarity: $D = 1 - \exp(-\eta)$

5. Classical MDS on the landmark dissimilarity matrix + Nyström extension for all pixels

6. Map 3 MDS dimensions to RGB channels

The result is a 3-band GeoTIFF where similar colours indicate similar predicted community composition.

10 Identifiability considerations and practical safeguards

A central challenge in the ObsPair formulation is that the between-site match probability depends on both richness and turnover:

$$p_{ij} = S_{ij} \cdot \frac{\alpha_i + \alpha_j}{2\alpha_i\alpha_j}, \qquad S_{ij} = \exp(-\eta_{ij}). \tag{37}$$

Consequently, a low between-site match probability can be explained either by low similarity (β-driven turnover) or by high effective richness at one or both sites (α-driven dilution of the chance of a match). This creates a genuine identifiability problem unless the two components are anchored by different sources of information or are constrained to play distinct structural roles.

10.1 Primary source of identifiability

The strongest direct information for α_i comes from within-site pairs, for which

$$p_{ii} = \frac{1}{\alpha_i}. \tag{38}$$

This implies that within-site pairs are the main empirical anchor for the richness model, whereas between-site pairs are responsible for learning turnover conditional on that baseline. In practice, identifiability improves when the fitting procedure preserves this division of labour. Useful strategies include:

- assigning sufficient weight to within-site pairs during training,
- pretraining the α model on within-site pairs before joint optimisation,
- freezing or partially damping α while the β model is first introduced,
- and using auxiliary constraints on α based on observed richness or sample-completeness information when available.

10.2 Constraining the turnover baseline

A further safeguard is to prevent the β model from learning a free baseline reduction in match probability that competes with α. In particular, if turnover is parameterised as

$$\eta_{ij} = \eta_0 + f(\mathbf{\Delta}_{ij}), \tag{39}$$

then the intercept η_0 can absorb residual mismatch even when environmental distance is zero, weakening the separation between richness and turnover. A cleaner formulation enforces

$$\eta(\mathbf{0}) = 0, \tag{40}$$

so that turnover only explains additional decay in similarity with increasing environmental separation. This is consistent with the ecological interpretation that identical environmental conditions should not imply positive turnover in the modeled distance component.

10.3 Controlling flexibility asymmetrically

Because α and β both influence between-site match probabilities, identifiability is sensitive to their relative flexibility. In practice, the α model is often the more dangerous absorber of residual structure, because it contributes to both within-site and between-site probabilities and can mimic unmodeled heterogeneity. It is therefore often beneficial to keep the α model somewhat more conservative than the β model, for example by:

- using a smaller network for α than for β,
- applying stronger weight decay or dropout to α,
- shrinking interaction terms more strongly than main effects,
- and restricting any spatial residual term in α to be broad-scale and heavily penalised.

The goal is not to underfit richness, but to avoid allowing α to become a universal explanation for all pairwise mismatch.

10.4 Separating spatial and environmental structure

Spatial structure is a particularly important source of confounding because both richness and compositional turnover are typically spatially autocorrelated. If flexible spatial residual structure is allowed in both the α and β components, the two sub-models may compete to explain the same latent gradient. Identifiability is improved by assigning spatial structure more deliberately, for example by:

- placing residual spatial structure in α only and keeping β purely environmental,
- placing geographic separation in β only and keeping α non-spatial,
- or restricting the two components to operate at different spatial scales (e.g. broad-scale residual structure in α, local environmental turnover in β).

In all cases, the preferred strategy is to avoid giving both sub-models unrestricted access to the same latent spatial signal.

10.5 Observation-process covariates and auxiliary information

Under the equal-draw assumption, α should be interpreted as an *effective richness under the recording process*, not necessarily a direct estimate of true species richness. This means that sampling effort, detectability, checklist completeness, and species-evenness effects can all enter the α estimate. If these recording-process components are not modeled explicitly, they may be confounded with turnover.

As described in Section 5.4, the current implementation addresses this directly through an additive effort decomposition:

$$\alpha_i = \mathrm{softplus}\big(f_\alpha(\mathbf{z}_i) + f_\omega(\mathbf{w}_i)\big) + 1, \tag{41}$$

where six spatial proxies for observation effort (ALA record density, institutional proximity, road access, etc.) are modeled through a dedicated EffortNet sub-network. This allows the environmental component f_α to focus on true ecological variation while the effort component f_ω absorbs recording-process artefacts. At prediction time, zeroing the effort logit recovers an estimate of richness driven purely by environmental conditions.

The additive structure on the logit scale means that effort acts as a multiplicative scaling of the match probability: sites with high recording effort will have higher apparent richness (and therefore lower within-site match probability), which is consistent with the intuition that better-sampled sites reveal more species under the equal-draw framework.

Where richer site-level biodiversity summaries are available, a further improvement is to include an auxiliary loss or penalty that encourages α to remain consistent with externally estimated richness-like quantities.

10.6 Pair design and training strategy

Identifiability is also influenced by which pairs are seen during fitting. Pairs are most informative when they help break the correlation between geography, environment, and expected turnover. The hex-balanced 3-tier sampling scheme (Section 3) is designed to address this directly:

- **Same-hex pairs** (Tier 1, 50%) provide geographically close pairs spanning small environmental distances — these anchor the near-zero end of the turnover response curves.
- **Neighbour-hex pairs** (Tier 2, 30%) span moderate ($\sim$2–4°) distances, capturing regional turnover where environmental gradients begin to diverge.
- **Distant-hex pairs** (Tier 3, 20%) span continental distances including environmentally similar but geographically distant site pairs, which are critical for separating distance decay from environmental turnover.
- **Within-site pairs** provide sufficient repeated within-site information to stabilise α, with equal quotas per hex ensuring geographic coverage.

The hex-balanced design also ensures that data-sparse regions contribute meaningfully to training rather than being overwhelmed by densely sampled urban areas. The IPW correction (Section **??**) restores population-level correctness so that the fitted model reflects the true ecological landscape rather than the geography of recording effort.

From an optimisation perspective, staged fitting further improves stability. The implemented sequence is:

1. fit α primarily from within-site pairs and richness constraints (cyclic alpha phase),
2. fit β conditional on that α baseline (cyclic beta phase),
3. repeat alternating alpha/beta cycles until convergence,
4. and introduce geographic distance as a beta input only after the environmental model is stable (Phase 2 fine-tuning).

This does not change the statistical model, but helps preserve the intended functional roles of the two components during training.

10.7 Summary

In short, the α and β components become more identifiable when they are forced to answer different questions from different evidence sources:

- α should primarily explain within-site matching and site-level richness/recording-process variation,
- β should primarily explain the decay in similarity with between-site environmental separation,
- and residual spatial or observational effects should be assigned deliberately rather than left for both components to absorb opportunistically.

These considerations are important regardless of whether the sub-models are specified using splines, monotone neural networks, or lattice-style architectures.

11 Further thoughts

- All the interpretive considerations from the TMB document apply: α represents "effective richness under the recording process" rather than true species counts, and the equal-draw assumption means our estimates conflate richness with evenness.
- The additive monotone network can be interrogated post-hoc by sweeping each environmental dimension while holding others at zero, producing per-variable response curves analogous to the I-spline partial functions in GDM. The factored network additionally allows examination of joint response surfaces.
- The ability to capture variable interactions (factored beta) is a double-edged sword — more flexible, but harder to interpret. Careful examination of joint response surfaces is recommended. The additive beta is preferred when interpretability is paramount.

- The effort/detectability decomposition (Section 5.4) provides a principled way to separate observation-process artefacts from ecological signal in the alpha model. The effort logit surface is a useful diagnostic: regions where $f_\omega > 0$ indicate that recording effort inflates apparent richness, while $f_\omega < 0$ indicates under-sampled areas. Comparing the environment-only and full alpha surfaces quantifies the magnitude of the sampling-effort correction.

- The network's richness predictions should be compared against the TMB P-spline predictions on the same data to assess consistency between the two approaches.

- If uncertainty quantification is needed (which the TMB `sdreport` provides), options include MC dropout at prediction time, deep ensembles, or training multiple models with different random seeds.

- The data pipeline (Steps 1–4 of the R pipeline) is shared between both approaches, so switching between TMB and NN fitting requires only an export step (`export_for_nn.R`) and a single Python command.

- The hex-balanced 3-tier sampling scheme (Section 3) ensures geographic coverage across the continent while the IPW weights and retrospective Bernoulli correction (Section **??**) keep the loss surface well-conditioned. The sampler produces a diagnostic PDF summarising geographic balance, tier breakdowns, weight distributions, and ESS.

- The default training mode is `cyclic_finetune`, which combines damped cyclic block-coordinate descent (Phase 1) with a joint or geographic fine-tuning pass (Phase 2), followed by a short calibration phase. This most closely mirrors the iterative R pipeline (`clesso_iterative.R`) while adding principled co-adaptation and probability calibration.

- The observed-richness lower-bound penalty should use *corrected* S_{obs} values (true species counts from raw data) rather than the subsampled pair-source richness, which underestimates true counts.

12 Thing to try

- Add an auxiliary loss to encourage alpha predictions to align with observed richness at the training sites, using a soft penalty that allows some deviation but discourages extreme mismatches.

- Confirm use of train-test split, look to see if we have done this incorrectly, make sure it"s maintaining the spatial structure of the data (e.g. no leakage of nearby sites between train and test).

- Try training with and without the beta LR multiplier and gradient amplification to see how much they help with the gradient imbalance issue.

- Experiment with different cyclic training schedules (e.g. varying N_α and N_β) to find a good balance between allowing alpha to adapt and giving beta enough time to learn.
- Try the "geo" fine-tuning mode to see if adding geographic distance as a beta input improves the fit, and check the resulting geographic response to ensure it is interpretable and not just absorbing residual noise.
- Experiment with different hard-pair mining strengths (λ_{hm}) and binning strategies to see how they affect convergence speed and final performance.
- Look at temperature / platt scaling calibration instead of the full tuning happening now
- Reduce alphanet complexity (e.g. smaller network, stronger regularisation) to see if it helps with identifiability and prevents it from absorbing turnover signal.
- Go back to additive betanet and see if it can learn a reasonable fit without the beta LR multiplier and gradient amplification, especially with the retrospective correction in place.
- Explore adding stopping on blocked AUC or other classification metrics in addition to the validation loss, to ensure the model is not just improving likelihood but also predictive performance on the match/mismatch task.
- Need to work on the stratum weighting approach to ensure weights calculated are as expected. In particular make sure that design weight track through from sampling to estimation
- Need to brainstorm completeness model approach. Model is conceptually neat, I think there is more informaiton we could include into the completeness section (e.g.n unique visits, repeated sampling, n unique observers, unique species vs observation fraction, triplets?, systematic survey identifier, n 'sites' within hex [or radius])
- Look more closely at the standardisation of ecological distances in the betanet - could be masking effect
- When Δ env $= 0$, η also $= 0$ might want to revisit this given model doesn't capture all sources of variation in a community.
- Additive betanet initialisation current initialises at $\eta = 5$, look at what $eta = 5$ translates to, we might want ot reduce this
- We want eta to range between 0-2, higher effectively gives total dissimilarity. Need to work on betanet to pull it down. This will affect eta collapse controls and many other things. η collapse mechanism pushes η to be above 1 which is approx. 0.3 dissimilarity so we loose the finer gradient.

- Put some thought into seperating the covariate set. Do the same things matter for alpha and beta?!?!
- Play around with regularisaiton of alphanet, increase drop out, stronger λ on interaction weights, shallower network, to see if it helps with identifiability and prevents alpha from absorbing turnover signal.
- Compare pair-level splitting versus *site-first* splitting (generate pairs only after splitting sites), to confirm that no site-level leakage is inflating validation performance.
- Try a stricter *blocked spatial cross-validation* design (e.g. hold out whole hexes / neighbouring hex clusters / bioregions) and compare AUC against the current validation scheme.
- Save and compare checkpoints *before* and *after* the calibration phase to test whether the calibration step is improving calibration but damaging ranking performance (AUC).
- Stop on *blocked validation AUC* rather than only the weighted validation loss, and compare the selected checkpoint to the current best-loss checkpoint.
- Report AUC separately for strata W, S, N, D, and also by geographic region / held-out block, to identify whether the performance drop is concentrated in particular pair types or regions.
- Remove the joint fine-tuning stage and compare against the best Phase 1 cyclic checkpoint, to see whether the later co-adaptation of alpha and beta is undoing the separation learned during cyclic training.
- Freeze alpha during Phase 2 fine-tuning and train only beta, then compare against joint fine-tuning to test whether alpha is re-absorbing turnover structure late in training.
- Try training with the anti-collapse regulariser turned off, reduced, or annealed to zero late in training, to see whether it is preventing η collapse early but hurting generalisation once training stabilises.
- Train with no hard-pair mining at all, and compare against mined runs, to test whether mining is improving optimisation speed at the cost of worse generalisation.
- Train with match-boost excluded, included only in warm-up, and included throughout, to check whether rare-match shaping is helping beta learn or instead biasing the representation.
- Try removing the retrospective correction entirely in a simple ablation, or using a coarser / finer version of the retention-rate correction, to test whether the current correction is stabilising training or introducing mismatch with the true sampling process.

- Check whether the retention rates $r_{s,0}, r_{s,1}$ should vary at finer resolution than just stratum (e.g. by tier, region, or proposal bucket), and test whether this improves fit.
- Experiment with *uniform within-hex site sampling* instead of m_i^2-weighted within-site sampling, to see whether the current proposal is making alpha too dependent on observed support size.
- Compare the current between-site design weights against a simpler site-pair-weighted target, to test whether the exact weighting scheme is part of the validation–test gap.
- Try training with different stratum weights $\boldsymbol{\lambda}$, especially increasing the weight on distant pairs or within-site pairs, to see how sensitive fit and test AUC are to the composite-likelihood balance.
- Remove Fourier positional encoding from alpha, and separately remove geographic distance from beta, to test which spatial pathway is carrying most of the apparent predictive signal.
- Try keeping geography in only one sub-model at a time (alpha-only spatial, beta-only spatial, neither), to check whether the two components are competing to explain the same spatial gradient.
- Compare additive versus multiplicative effort models, and also a run with `use_effort=False`, to test whether the effort pathway is improving realism or simply adding variance / identifiability problems.
- Make EffortNet linear or very shallow, to see whether a simpler effort correction generalises better than the current flexible effort network.
- Reduce beta complexity as well as alpha complexity (e.g. fewer hidden units per monotone function, shallower factored interaction network) to see whether the current beta model is fitting validation idiosyncrasies.
- Add a small auxiliary *ranking loss* (or other AUC-oriented surrogate) alongside the corrected BCE, to test whether optimising likelihood alone is misaligned with the evaluation metric.
- Check batch-size sensitivity (smaller versus larger batches), because very large batches can reduce gradient noise but sometimes worsen generalisation.
- Run multiple random seeds for the same configuration and quantify variability in validation and test AUC, to determine whether the current performance gap is partly due to optimisation variance rather than systematic model failure.
- Try a simple seed ensemble of the best few models, to see whether averaging predictions improves test AUC and stabilises the ranking.

- Compare the neural network fit against a deliberately simpler baseline (e.g. additive beta + small alpha net, or even a spline / shallow model) to establish whether the current extra flexibility is actually buying test performance.
- Look into chelsa and terraclimate covariates as alternatives to AWAP, to see whether the choice of climate dataset is influencing the fit and generalisation.

12.1 Habitat condition decomposition for alpha

A useful extension is to separate *potential ecological richness* from *realised richness under current habitat condition*. The intuition is that climate, substrate, and other environmental predictors may describe the richness a site could support under broadly intact native condition, while land use, habitat loss, fragmentation, and other pressure variables reduce the proportion of that richness that remains.

To represent this, we decompose richness into three components:

- α_i^{pot}: potential native richness under environmental conditions,
- $c_i^{\text{cond}} \in [0, 1]$: condition-retention factor, interpreted as the proportion of potential native richness retained under current habitat condition,
- α_i^{ecol}: ecological richness after condition effects are applied.

The potential richness component is modelled using the existing environmental alpha network:

$$\alpha_i^{\text{pot}} = 1 + \text{softplus}(f_\alpha(\mathbf{z}_i; \boldsymbol{W}_\alpha)) \tag{42}$$

where $\mathbf{z}_i$ contains environmental and substrate covariates and the softplus+1 transform ensures $\alpha_i^{\text{pot}} > 1$.

Habitat condition is modelled through a separate bounded network:

$$c_i^{\text{cond}} = \sigma(f_{\text{cond}}(\mathbf{q}_i; \boldsymbol{W}_{\text{cond}})) \tag{43}$$

where $\mathbf{q}_i$ is a vector of habitat condition / human pressure covariates (e.g. land use intensity, vegetation loss, fragmentation, human footprint, grazing pressure), $\sigma(\cdot)$ is the logistic sigmoid, and therefore $c_i^{\text{cond}} \in (0, 1)$.

The resulting ecological richness is then:

$$\alpha_i^{\text{ecol}} = 1 + c_i^{\text{cond}} \left(\alpha_i^{\text{pot}} - 1\right). \tag{44}$$

This formulation scales the *excess above the minimum richness bound of 1*, so that:

- $c_i^{\text{cond}} \approx 1$ implies little or no condition-related loss,
- $c_i^{\text{cond}} < 1$ implies proportional reduction in retained native richness,
- and $c_i^{\text{cond}} \to 0$ collapses richness toward the lower bound of the model.

Interpretation. The condition-retention factor is best interpreted as the *proportion of environmentally expected native richness retained under current habitat condition*, rather than as a literal measure of total biodiversity intactness. This keeps the quantity closely tied to the alpha component of the ObsPair likelihood.

Relationship to effort/completeness. When an observation-process decomposition is also used, it is natural to apply effort *after* condition:

$$\alpha_i^{\text{eff}} = 1 + c_i^{\text{det}} \left(\alpha_i^{\text{ecol}} - 1\right) \tag{45}$$

where c_i^{det} is a completeness / detectability factor. In this hierarchy:

$$\alpha_i^{\text{pot}} \;\rightarrow\; \alpha_i^{\text{ecol}} \;\rightarrow\; \alpha_i^{\text{eff}}$$

the environment determines potential richness, habitat condition reduces the proportion of native richness retained, and the observation process determines how much of that ecological richness is expressed in the observed draw process. In simpler implementations, the effort component can be omitted initially and the pair likelihood can be written directly in terms of α_i^{ecol}.

Regularisation. Because c_i^{cond} is a flexible multiplicative correction, a shrinkage penalty toward intact condition is useful:

$$\mathcal{R}_{\text{cond}} = \lambda_{\text{cond}} \cdot \frac{1}{|B|} \sum_{i \in B} \left(1 - c_i^{\text{cond}}\right)^2 . \tag{46}$$

This encourages the model to explain richness variation through the environmental alpha pathway unless the data support a condition-related reduction.

Advantages. This decomposition has several appealing properties:

1. it gives habitat condition a clear ecological interpretation as a bounded retention factor rather than an unrestricted additive correction,
2. it separates broad environmental potential from anthropogenic degradation,
3. it produces a directly mappable condition surface c_i^{cond},
4. and it can reduce the burden on the main alpha network to absorb land-use and pressure effects indirectly.

Potential limitations. The main caution is confounding between environment, habitat condition, and the observation process. ConditionNet should therefore be supplied only with covariates that genuinely describe habitat state or anthropogenic pressure, and its flexibility should be kept more restricted than the main environmental alpha network.

12.2 Elastic distance weighting for cyclic optimisation

A soft extension of the within-site / between-site separation is to allow the contribution of between-site pairs to vary smoothly with pair distance during optimisation. The underlying probabilistic model is *not* changed:

$$p_{i,j} = \exp(-\eta_{i,j}) \cdot \frac{\alpha_i + \alpha_j}{2\alpha_i\alpha_j}. \tag{47}$$

Instead, distance-dependent weights are used only to control how strongly different pairs contribute during the alpha and beta optimisation phases.

Pair distance. Let $d_{i,j}$ denote a scalar pair distance, defined either in geographic space or environmental space. For example, geographic distance may be used:

$$d_{i,j}^{\text{geo}} = \text{Haversine}(i, j), \tag{48}$$

or a standardised environmental distance:

$$d_{i,j}^{\text{env}} = \left(\sum_{k=1}^{m} \frac{(x_{k,i} - x_{k,j})^2}{\sigma_k^2} \right)^{1/2}. \tag{49}$$

Within-site pairs satisfy $d_{i,i} = 0$.

Elastic alpha and beta weights. We define a smooth alpha-weighting function $\omega_\alpha(d) \in [0, 1]$ that decreases with distance, and a complementary beta-weighting function:

$$\omega_\beta(d) = 1 - \omega_\alpha(d). \tag{50}$$

A convenient default is the logistic form:

$$\omega_\alpha(d) = \frac{1}{1 + \exp\left(\frac{d-\tau}{s}\right)}, \qquad \omega_\beta(d) = 1 - \omega_\alpha(d), \tag{51}$$

where:

- τ is the transition distance at which alpha and beta contributions are equal,
- s controls the smoothness of the transition.

Thus:

- pairs with $d \ll \tau$ contribute mostly to alpha,
- pairs with $d \gg \tau$ contribute mostly to beta,
- and within-site pairs ($d = 0$) remain effectively pure alpha information.

Key point. These weights affect only *how training signal is allocated* during block-coordinate optimisation. They do *not* alter the generative pairwise match probability. This preserves the interpretation of the fitted model while allowing nearby and distant pairs to play different roles during optimisation.

12.2.1 Alpha phase

In the current implementation, the alpha phase is fitted primarily or exclusively on within-site pairs, for which $d = 0$ and $\omega_\alpha(d) = 1$. In that strict formulation, the elastic weighting does not materially change the alpha phase. However, if between-site pairs are later allowed to contribute weakly to alpha learning, the alpha objective can be written as:

$$\mathcal{L}_\alpha = \frac{\sum_{p \in P} d_p \, \omega_\alpha(d_p) \, \ell_p}{\sum_{p \in P} d_p \, \omega_\alpha(d_p)} + \lambda_{\text{lb}} \mathcal{R}_\alpha + \lambda_{\text{anchor}} \mathcal{R}_{\text{anchor}}, \tag{52}$$

where d_p is the design weight for pair p, d_p in $\omega_\alpha(d_p)$ denotes the corresponding pair distance, and ℓ_p is the per-pair Bernoulli loss (optionally with retrospective correction).

12.2.2 Beta phase

The main practical use of the elastic weighting in the current formulation is in the beta phase, where alpha is frozen and beta is trained on between-site pairs. The beta objective becomes:

$$\mathcal{L}_\beta = \frac{\sum_{p \in P_{\text{between}}} d_p \, \omega_\beta(d_p) \, \ell_p}{\sum_{p \in P_{\text{between}}} d_p \, \omega_\beta(d_p)} + \lambda_\eta \mathcal{R}_\eta + \lambda_{\text{boost}} \mathcal{L}_{\text{boost}}. \tag{53}$$

This gives low-distance between-site pairs relatively less influence on beta updates, while allowing increasingly distant pairs to dominate turnover learning. In this sense the method provides a soft continuum between the strict within-site / between-site split and a fully joint pairwise likelihood.

Interpretation. Under this formulation:

- within-site pairs remain the strongest direct anchor for alpha,
- nearby between-site pairs are treated as primarily informative about richness and only secondarily about turnover,
- and increasingly distant pairs are treated as progressively more informative about beta diversity.

Choice of distance metric. Geographic distance is the simplest and safest choice for the elastic weighting because it is exogenous to the turnover covariates and less likely to create circularity. Environmental distance is more ecologically targeted, but may blur the same signal that beta is intended to learn.

Learning the weighting curve. Rather than fixing (τ, s) a priori, they can be treated as outer-loop optimisation parameters. In that case, the shape of the elastic weighting function is learned between training cycles, while the alpha and beta sub-model parameters are optimised conditionally on the current weight curve. This preserves the interpretation of the model itself while allowing the balance between near-pair and far-pair influence to adapt to the data.

Advantages. The elastic weighting scheme may help:

1. reduce competition between alpha and beta during optimisation,
2. generalise the hard within-site / between-site split into a smooth continuum,
3. and allow the model to exploit nearby between-site pairs without forcing them to behave identically to distant turnover pairs.

Potential limitations. The weighting function introduces additional tuning parameters and may suppress true local turnover if the transition is too aggressive. It should therefore be treated initially as an optimisation device rather than a change to the ecological model itself.

12.3 Transform-first monotone network with controlled interactions (`beta_type=``transform_interaction''`)

The transform beta network described in Section 6.6 is additive across environmental dimensions:

$$\eta_{i,j}^{\text{main}} = \sigma_{\text{clamp}}\left(\sum_{k=1}^{m}\left[g_k(|T_k(\tilde{x}_{k,i}) - T_k(\tilde{x}_{k,j})|) - g_k(0)\right]\right), \tag{54}$$

where each T_k is a learned monotone transform of the raw environmental axis and each g_k is an origin-anchored monotone weighting function. This architecture is the closest neural analogue of GDM's I-spline parameterisation, but by construction it cannot represent cross-dimension interactions. Section 6.5 addresses this limitation for the distance-input architecture via a shared monotone interaction network. Here we introduce the analogous extension for the transform-first architecture. :contentReference[oaicite:3]index=3 :contentReference[oaicite:4]index=4

Let

$$d_{k,ij}^{(T)} = |T_k(\tilde{x}_{k,i}) - T_k(\tilde{x}_{k,j})| \tag{55}$$

denote the transformed separation along environmental dimension k, and define the additive per-dimension contribution

$$s_{k,ij} = g_k\left(d_{k,ij}^{(T)}\right) - g_k(0). \tag{56}$$

The additive transform model then contributes

$$z_{ij}^{\text{main}} = \sum_{k=1}^{m} s_{k,ij}. \tag{57}$$

To allow controlled departures from strict additivity, we introduce a small monotone interaction head operating on low-dimensional encodings of the transformed separations. For each dimension k, define

$$\mathbf{h}_{k,ij} = e_k\left(d_{k,ij}^{(T)}\right) \in \mathbb{R}^{d_{\text{enc}}}, \tag{58}$$

where $e_k : \mathbb{R}_{\geq 0} \to \mathbb{R}_{\geq 0}^{d_{\text{enc}}}$ is a shallow monotone encoder with non-negative weights and monotone activations. The interaction input is the concatenation

$$\mathbf{h}_{ij} = [\mathbf{h}_{1,ij}; \ldots; \mathbf{h}_{m,ij}] \in \mathbb{R}^{m d_{\text{enc}}}. \tag{59}$$

A shared monotone interaction network $\phi(\cdot)$ then produces a scalar interaction correction,

$$z_{ij}^{\text{int}} = \phi(\mathbf{h}_{ij}) - \phi(\mathbf{0}), \tag{60}$$

where baseline subtraction ensures that the interaction contribution is exactly zero when all transformed separations are zero. The total turnover predictor is

$$\eta_{i,j} = \sigma_{\text{clamp}}\left(z_{ij}^{\text{main}} + \lambda_{\text{int}}\, z_{ij}^{\text{int}}\right), \tag{61}$$

with interaction strength controlled by a scalar $\lambda_{\text{int}} \geq 0$.

Monotonicity and ecological interpretation. Because each T_k, g_k, and e_k is monotone, and ϕ is a monotone network with non-negative weights receiving non-negative inputs, the full model remains monotone in each transformed separation $d_{k,ij}^{(T)}$. Thus increasing environmental separation along any dimension cannot decrease predicted turnover. The additive component retains the same interpretation as the original transform architecture: a sum of per-dimension, GDM-like transformed-distance contributions. The new interaction term captures residual joint effects such as the impact of temperature separation depending on the magnitude of precipitation separation.

Why the interaction is "controlled". The interaction head is deliberately restricted in four ways. First, it acts as a residual correction on top of the additive transform model rather than replacing it. Second, it is baseline-anchored so that $\eta(\mathbf{0}) = 0$ is preserved in the modeled environmental component. Third, the interaction encoder width d_{enc} and the depth of ϕ are kept small. Fourth, the scalar λ_{int} shrinks the interaction term relative to the main additive signal.

Recommended default settings. A practical default is $d_{\text{enc}} = 4$ or 8, one hidden layer per encoder, one hidden layer in ϕ, and $\lambda_{\text{int}} \in [0.05, 0.2]$. In optimisation, the additive transform pathway should be trained first, with the interaction head either frozen or heavily shrunk during the initial beta-only phase, then unfrozen during joint fine-tuning. This preserves the division of labour between the interpretable main effects and the weaker interaction correction.

Optional regularisation. To further stabilise identifiability, the interaction correction can be penalised directly:

$$\mathcal{R}_{\text{int}} = \lambda_\phi \|\theta_\phi\|_2^2 + \lambda_z \cdot \frac{1}{|P_{\text{btw}}|} \sum_{(i,j)\in P_{\text{btw}}} \left(z_{ij}^{\text{int}}\right)^2, \tag{62}$$

where θ_ϕ denotes the parameters of the interaction head. The first term discourages a highly flexible interaction network; the second discourages the interaction correction from dominating the additive signal unless supported by the data.

Interpretability trade-off. Compared with the pure transform model, Equation 61 sacrifices some per-dimension interpretability in exchange for the ability to represent cross-dimension effects. However, because the additive transform term remains explicit and dominant, partial-response visualisation of the learned T_k and g_k functions remains meaningful. The interaction term should therefore be viewed as a structured residual that augments, rather than replaces, the GDM-like decomposition.